



Previsão de Séries Temporais com Modelos SARIMA: Um Estudo sobre o Número Semanal de Jogadores de Counter-Strike 2

Letícia Yumi Ichibara - RA: 834396

Marcelo Xinhong Huang - RA: 832111

Pedro Henrique de Araujo - RA: 831235

Universidade Federal de São Carlos - Departamento de Estatística

Julho de 2026

Enunciado: “Apresentar uma análise de dados real”

Sumário

1	Introdução	3
2	Análise exploratória	4
2.1	Gráfico da série	4

2.2	Estatísticas descritivas	4
2.3	Decomposição STL	5
2.4	Gráficos ACF e PACF	6
3	Estacionariedade	6
3.1	Testes na série original	7
3.2	Diferenciação de ordem 1	8
4	Detecção de outliers	9
5	Modelagem	10
5.1	Seleção automática com <code>auto.arima</code>	11
6	Diagnóstico dos resíduos	12
6.1	Gráficos de diagnóstico usando a função “ <code>checkresiduals</code> ”	12
6.2	Teste de Ljung-Box	13
6.3	Normalidade dos resíduos	13
6.4	Homocedasticidade	14
7	Previsão	15
8	Validação (treino/teste)	16
8.1	Métricas de erro	16
9	Modelos alternativos: Alisamento exponencial de Holtwinters e de ETS	17
10	Conclusão	22
	Referências	23
11	Apêndice	24

Resumo

Este trabalho apresenta uma análise de séries temporais aplicada ao número semanal de jogadores ativos de Counter-Strike 2 (CS2), no período de janeiro de 2016 a dezembro de 2025, com dados extraídos da plataforma SteamDB. A metodologia envolveu análise exploratória, decomposição STL, testes de estacionariedade (ADF e KPSS), detecção de outliers via tsoutliers, identificação e ajuste de um modelo SARIMA, tanto pela leitura dos gráficos ACF/PACF quanto pela função `auto.arima`, além de diagnóstico dos resíduos e validação preditiva por meio de janela treino/teste. Os resultados indicaram uma série com tendência crescente e sazonalidade, tornada estacionária após uma diferenciação regular, e a presença de outliers associados a eventos relevantes do jogo, como a pandemia de COVID-19 e o BLAST.tv Paris Major 2023. O modelo $SARIMA(1, 1, 2)(0, 0, 1)_{52}$ foi selecionado como o mais adequado, apresentando resíduos sem autocorrelação significativa, embora com desvios de normalidade e homocedasticidade. A validação com dados de 2025 resultou em um MAPE de 5.91%, indicando boa capacidade preditiva do modelo.

Palavras-chave: Séries temporais. Modelo ARIMA/SARIMA. Previsão. Counter-Strike 2.

1 Introdução

A análise de séries temporais é um método estatístico bem eficiente para dados levantados ao longo de um determinado período de tempo, permitindo identificar padrões, tendências e sazonalidades.

Por isso, para esse trabalho, iremos realizar uma análise completa de uma série temporal, desde as medidas descritivas, decomposição STL, identificação de outliers até ajuste de modelos ARIMA e previsões.

A série escolhida para esta análise é o **número semanal de jogadores de Counter-Strike 2** (antes da transição para CS2 em 2023, era conhecido como CS:GO — *Counter-Strike: Global Offensive*), um jogo de tiro multiplayer da plataforma Steam, na qual Counter Strike 2 se encontra disponível para download. Os dados foram extraídos da plataforma de terceiros “SteamDB”, que registra o número de jogadores ativos por dia.

Para uma análise mais focada e precisa, a série foi reduzida/filtrada entre **1 de janeiro de 2016** e **31 de dezembro de 2025**, e os registros diários foram agregados em médias **semanais** (semanas iniciando às segundas-feiras).

2 Análise exploratória

Antes de qualquer modelagem, com a série apresentada, começaremos, agora, uma breve análise descritiva da série a fim de entender o comportamento visual da série de jogadores de Counter Strike 2 e caracterizar sua estrutura.

Para isso, iremos primeiramente visualizar a série, no período delimitado anterior.

2.1 Gráfico da série

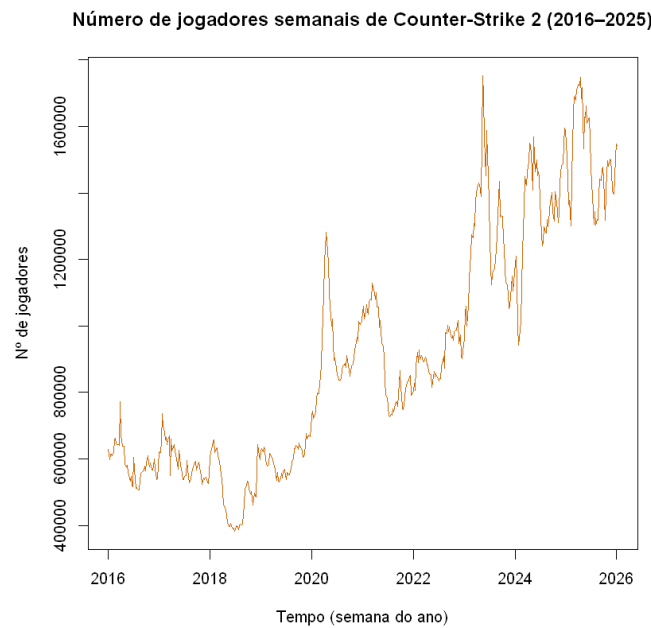


Figura 1: Série temporal do número semanal de jogadores

2.2 Estatísticas descritivas

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
382055	599685	849719	916332	1207915	1753420

Analisando visualmente a série, percebemos que ela possui uma tendência crescente bem evidente, com poucos picos decrescentes que mudem o crescimento da série. Esse comportamento se dá devido ao jogo ser multijogador, ou seja, não possui fases e mantém as pessoas jogando por mais tempo.

Além disso, há vários picos evidentes ao longo do tempo, provavelmente indicativos de eventos ou atualizações no jogo para aumentar o número de jogadores.

E note que a série não parece muito homocedástica, isso sugere que uma transformação pode ser necessária na modelagem que vamos abordar posteriormente.

2.3 Decomposição STL

Para verificar melhor a tendência, tanto quanto outros componentes da série, iremos decompô-la por meio da função `stl`, a qual decompõe a série em:

$$X_t = T_t + S_t + R_t, \quad (1)$$

em que T_t é o componente de tendência, S_t é o componente sazonal e R_t é o componente residual.

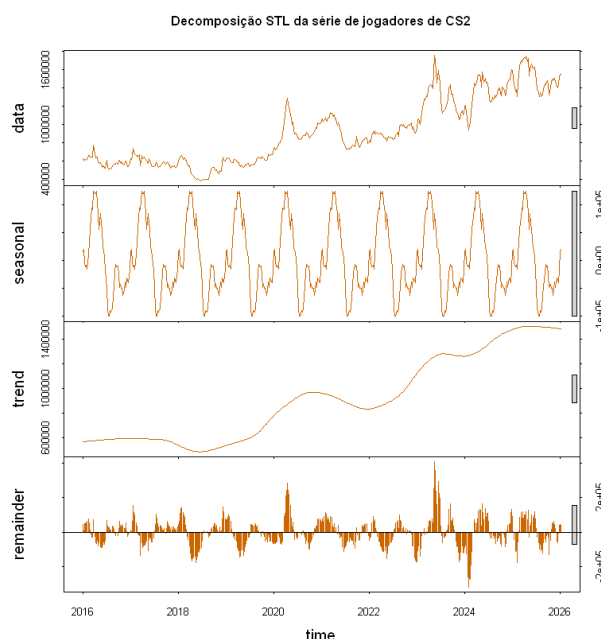


Figura 2: Decomposição STL da série

Com a série decomposta, podemos analisar seus componentes mais detalhadamente.

A **tendência** já age conforme o esperado, com um claro padrão crescente por todo o tempo analisado. Já a **sazonalidade** também é bem evidente, apresentando pico de jogadores em todo começo de ano, período de férias ou recesso, o que aumenta o número de jogadores.

Por fim, temos os **resíduos**, o restante da série após retirarmos a tendência e a sazonalidade. Exceto por picos em 2020 (pandemia) e 2023-2024 (lançamento

do CS2), os resíduos não apresentam padrão evidente, oscilando em torno de série com o passar do tempo.

2.4 Gráficos ACF e PACF

A última análise a ser feita nessa etapa será avaliar a presença de autocorrelação e autocorrelação parcial, ferramentas para identificar a dependência temporal da série.

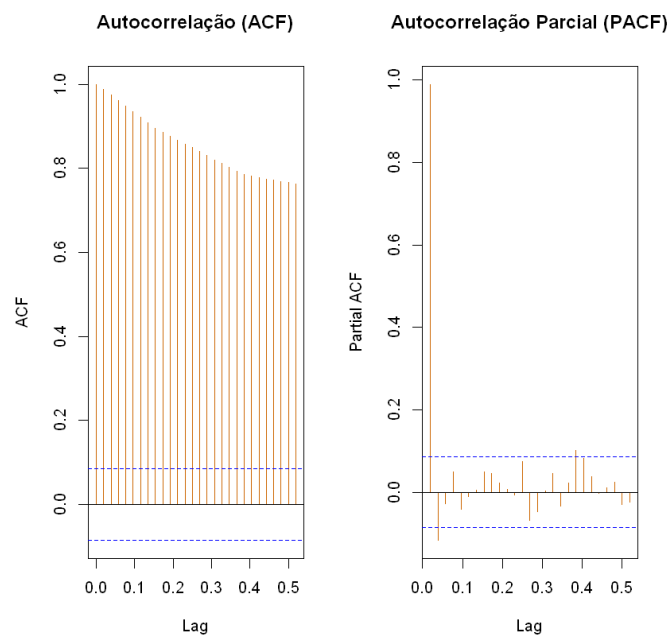


Figura 3: Gráficos ACF e PACF da série

Analisando a autocorrelação, há um indicativo forte de tendência, uma vez que os lags decaem de forma lenta, demorando a entrar no intervalo de confiança. Isso é uma evidência para diferenciarmos a série, passo que será feito na próxima etapa.

Já a autocorrelação parcial mostra um pico do lag 1 muito próximo de 1 decaindo rapidamente para dentro do intervalo de confiança logo no próximo lag, evidenciando que a série possa ser modelado futuramente como um modelo autoregressivo de ordem 1.

3 Estacionariedade

Como analisado na última etapa, é claro que a série utilizada não é estacionária, ou seja, sua média não se mantém constante ao longo do tempo.

Além da decomposição e gráfico ACF, uma maneira mais precisa de verificar estacionariedade é por meio de dois testes estatísticos. O teste Dickey-Fuller Aumentado (ADF) e o teste KPSS.

Os teste ADF e KPSS têm como hipóteses:

Teste	H_0	H_1
ADF (<i>Augmented Dickey-Fuller</i>)	Há raiz unitária (não-estacionária)	Não há raiz unitária
KPSS	Série estacionária	Série não-estacionária

3.1 Testes na série original

Realizando os testes:

Augmented Dickey-Fuller Test

```
data:  serie
Dickey-Fuller = -3.5364, Lag order = 8, p-value = 0.03881
alternative hypothesis: stationary
```

KPSS Test for Level Stationarity

```
data:  serie
KPSS Level = 6.286, Truncation lag parameter = 6, p-value = 0.01
```

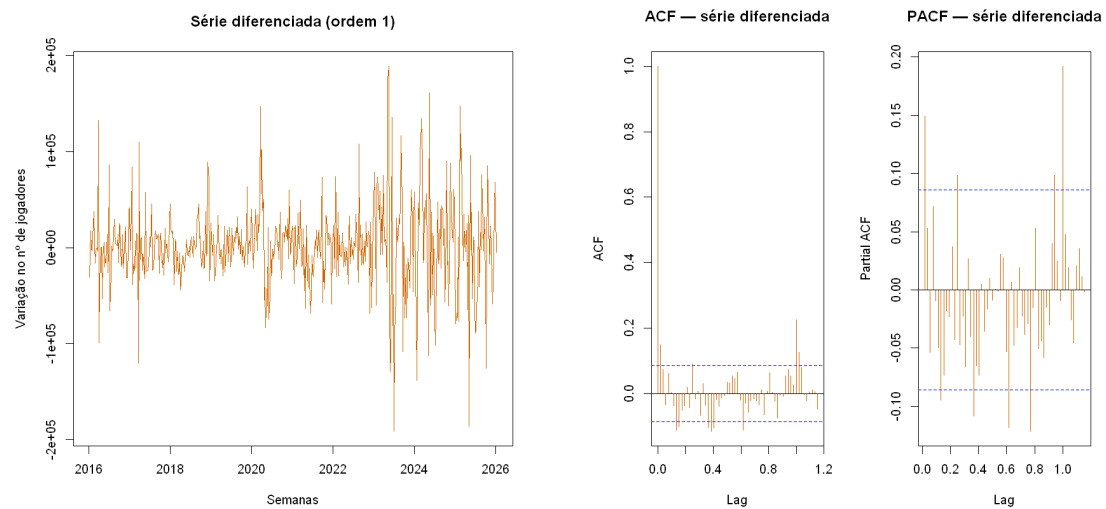
O p-valor do teste ADF resultou em aproximadamente 0,038, ou seja, rejeitamos H_0 e temos evidências que a série é estacionária.

Entretanto, o teste KPSS resultou em um p-valor menor que 0,01, ou seja, esse teste já indica que a série não é estacionária.

Dado a divergência dos testes e o fato dos gráficos anteriores indicarem com clareza a tendência crescente da série, o caminho é diferenciar a série e refazer os testes.

Iremos fazer, então, uma **diferenciação** de ordem 1 na série, isto é,

$$\Delta X_t = X_t - X_{t-1} \quad (2)$$



(a) Série diferenciada de ordem 1

(b) ACF e PACF da série diferenciada

Figura 4: Série diferenciada e suas funções de autocorrelação.

3.2 Diferenciação de ordem 1

A diferenciação de ordem 1 remove a tendência linear ao calcular:

Percebemos, analisando o gráfico da série diferenciada, que após uma diferenciação a série se tornou bem mais estacionária do que a anterior, com indícios de que a média se tornou constante.

Para confirmar a suspeita de estacionariedade, realizaremos os teste ADF e KPSS novamente, juntamente com os gráficos de ACF e PACF.

Augmented Dickey-Fuller Test

```
data: serie_diff
Dickey-Fuller = -8.6564, Lag order = 8, p-value = 0.01
alternative hypothesis: stationary
```

KPSS Test for Level Stationarity

```
data: serie_diff
KPSS Level = 0.043643, Truncation lag parameter = 6, p-value = 0.1
```

Uma vez que ambos os testes indicam que a série se tornou estacionária e o gráfico ACF (Figura 4b) mostra uma queda brusca do lag 0 ao lag 1, comportamento esperado em tendência constante, temos evidências suficientes para concluir que a série dos jogadores semanais de CS2 se torna estacionária após uma diferenciação.

4 Detecção de outliers

Antes de ajustar o modelo, identificamos possíveis outliers com a função `tso` do pacote `forecast`. Ela detecta automaticamente observações atípicas — como *Additive Outliers* (AO), *Level Shifts* (LS) e *Transient Changes* (TC), além de dizer qual é o impacto do outlier na série

Outliers:

	tipo	posicao	tempo	impacto	t estat
1	LS	220	2020:12	376735	13.995
2	TC	384	2023:20	248359	5.153
3	LS	391	2023:27	272742	8.353

Interpretação dos outliers:

Primeiro LS (posição 220 – Março/2020): A mudança permanente no nível da série coincide com o início das medidas de lockdown global decorrentes da pandemia de COVID-19. A classificação como Level Shift (e não como AO) é tecnicamente acertada, pois o isolamento social não gerou um pico passageiro, mas sim uma mudança estrutural no comportamento do consumidor, elevando de forma duradoura a base de espectadores e jogadores ativos devido à adoção massiva do entretenimento digital. A função `tso` nos diz que um impacto +376.735 jogadores, isso significa que a pandemia elevou o patamar de jogadores de forma permanente.

TC (Índice 384 – Maio/2023): Este ponto coincide com as finais do *BLAST.tv Paris Major 2023*, (*Major* é o maior campeonato oficial do jogo) o último Major da história do CS:GO antes da transição para o CS2. A classificação como Temporary Change é coerente com a natureza de um evento pontual de grande apelo midiático: observamos um pico expressivo de engajamento naquela semana específica, cujo efeito, embora intenso, dissipou-se gradualmente nas semanas seguintes, retornando à trajetória de equilíbrio anterior (ou à nova trajetória, como veremos a seguir). O impacto foi de +248.359 jogadores temporariamente.

Segundo LS (Índice 391 – Julho/2023): Este é o ponto mais interessante da análise. Aproximadamente dois meses após o Major, detectamos outro Level Shift positivo e permanente. Esse fenômeno pode ser interpretado como o “efeito legado” do campeonato. É plausível que o lançamento do CS2 (anunciado para o setembro de 2023) e o enorme sucesso do Major tenham atraído uma leva definitiva de novos jogadores e espectadores que, diferentemente do pico agudo de maio, se consolidaram na comunidade, elevando o patamar mínimo de atividade da série de forma estrutural. O impacto é +272.742 jogadores de forma permanente.

As estatísticas t associadas a cada outlier (variando de 5,15 a 13,99) superam amplamente o valor crítico usual ($|t| > 2$), confirmando a relevância desses choques para a dinâmica da série. Incluir esses efeitos apenas como “pontos a serem

monitorados” (como sugerido na análise preliminar) pode não ser suficiente. Posteriormente, na etapa de modelagem, uma opção é utilizarmos os parâmetros gerados pelo tso para construir variáveis de intervenção (dummies permanentes para os LS e com decaimento para o TC) que serão inseridas como regressoras externas (via argumento `xreg` da `auto.arima`). Isso poderia evitar que os outliers distorçam a estimativas dos parâmetros do modelo.

5 Modelagem

Após analisar os componentes da série, detectar os outliers e explorar os dados na escala original, podemos, enfim, ajustar um modelo apropriado para a nossa série temporal.

Como a série original apresentou indícios de heterocedasticidade, nós investigamos qual é o λ para usar na transformação de Box-Cox usando a função **BoxCox.lambda** do R, e o resultado é $\lambda = 0.3036$ que está entre 0 e 0.5. Com esse valor de λ e pelo princípio de parcimônia, decidimos usar a transformação logarítmica nos dados pois seria adequada. Ou seja: novo $X_t = X_t^* = \log(X_t)$

Além disso, note que, como a transformação não remove a tendência da série (apenas estabiliza a variância), o valor exato das estatísticas dos testes ADF e KPSS muda em relação à série original, porém a conclusão geralmente é a mesma e o número de diferenciações necessárias para remover a tendência continua o mesmo (nós verificamos isso para a série, refazendo os testes).

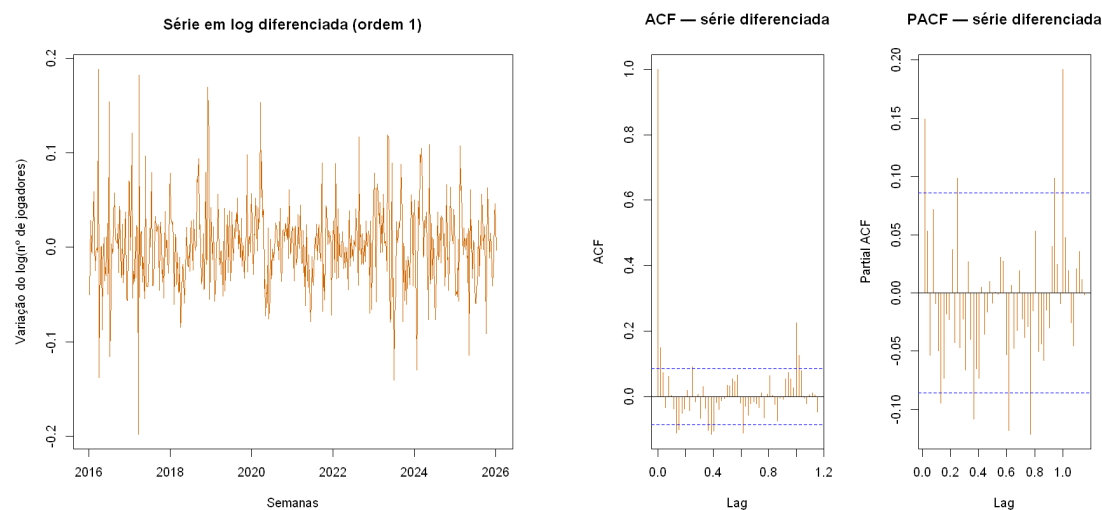
Podemos encontrar o modelo de duas maneiras diferentes: manualmente, analisando os gráficos ACF e PACF (Figura 5), ou pela função `auto.arima`, que ajusta o melhor modelo ARIMA encontrando aquele com o menor BIC, critério muito usado em seleção de modelos.

Em primeiro lugar, vale lembrar que esses gráficos ACF e PACF são obtidos após a diferenciação de ordem 1 da série. Logo, teremos $d = 1$ como parâmetro do modelo ARIMA.

Analisando o gráfico PACF, percebe-se que, após o lag 1, o próximo lag decai rapidamente para o intervalo de confiança. Com isso, um valor possível para o parâmetro autoregressivo seria $p = 1$.

Agora, pelo gráfico ACF, temos autocorrelações de pequena magnitude nos primeiros lags, mas percebe-se que o lag 1 apresenta-se para fora do intervalo de confiança e o lag 2 está em cima do tracejado, ou seja, no limiar do IC. Assim, um valor baixo para o parâmetro de médias móveis pode ser adequado, como $q = 1$ ou $q = 2$.

Porém, é válido perceber que ambos os gráficos apresentam um comportamento sazonal a cada um ano, de acordo com a escala transformada dos gráficos. Desse modo, uma diferença sazonal de 52 semanas, isto é, um ano, seria ideal para remover



(a) Série diferenciada de ordem 1

(b) ACF e PACF

Figura 5: log Série diferenciada e suas funções de autocorrelação.

a sazonalidade.

5.1 Seleção automática com `auto.arima`

Encontrando o melhor modelo ARIMA pelo função `auto.arima`, percebemos que aquele com menor BIC indica que

$$X_t^* \sim ARIMA(1, 1, 2)(0, 0, 1)[52] \quad (3)$$

é o melhor modelo, confirmando o ajuste manual pelos gráficos. Além disso, em R adicionamos um parâmetro "xreg" para considerar os efeitos de outliers.

Resumo do modelo:

Series: serie

Regression with ARIMA(1,1,2)(0,0,1)[52] errors

Coefficients:

	ar1	ma1	ma2	sma1	LS220	TC384	LS391
	-0.9522	0.9893	0.0229	0.2709	0.1594	0.1098	-0.1221
s.e.	0.0580	0.0729	0.0463	0.0460	0.0400	0.0371	0.0398

sigma^2 = 0.001722: log likelihood = 920.26

AIC=-1824.53 AICc=-1824.25 BIC=-1790.48

Training set error measures:

	ME	RMSE	MAE	MPE	MAPE	MASE
Training set	0.001278891	0.04117487	0.03009168	0.008875282	0.2204688	0.1518893

ACF1

Training set	-0.0009548183
--------------	---------------

Note que temos alguns coeficientes que são estatisticamente iguais a 0, e pelo princípio da parcimônia uma opção é removê-los do modelo. Porém, para o fim de obter melhores resultados na previsão, vamos prosseguir o modelo atual.

6 Diagnóstico dos resíduos

Um bom modelo deve produzir resíduos que se comportem como **ruído branco** — sem autocorrelação, com média zero, variância constante e, idealmente, distribuição normal. Para ver isso abaixo está uma figura de diagnóstico de resíduos.

6.1 Gráficos de diagnóstico usando a função “checkresiduals”

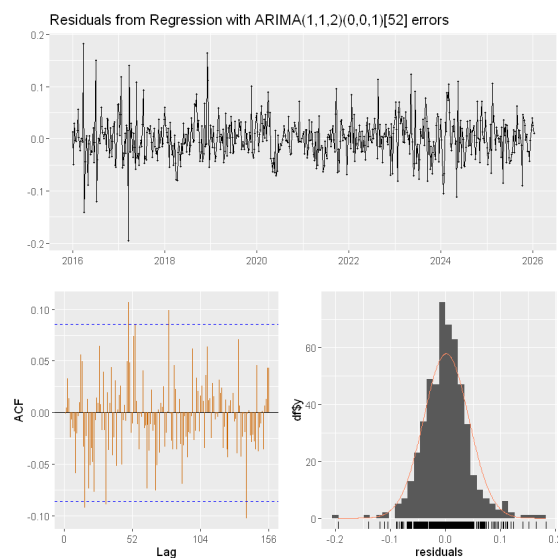


Figura 6: Diagnóstico dos resíduos do modelo ARIMA ajustado

Essa figura apresenta os principais diagnósticos dos resíduos do modelo ARIMA ajustado. O painel superior mostra a evolução temporal dos resíduos, permitindo verificar se permanecem distribuídos aleatoriamente em torno de zero. Observa-se que não há tendência ou sazonalidade remanescente, indicando que o modelo

conseguiu capturar a maior parte da estrutura sistemática da série. Entretanto, nota-se um aumento na dispersão dos resíduos em alguns períodos, principalmente entre 2023 e 2025, sugerindo a ocorrência de maior volatilidade nesse intervalo.

O gráfico da Função de Autocorrelação (ACF), localizado no canto inferior esquerdo, mostra que a maior parte das autocorrelações encontra-se dentro das bandas de confiança de 95%. Embora existam alguns lags isolados que ultrapassem ligeiramente esses limites, não se observa um padrão persistente de autocorrelação, indicando que os resíduos apresentam comportamento próximo ao de ruído branco.

Por fim, o histograma dos resíduos, acompanhado da curva normal ajustada, evidencia que a distribuição é aproximadamente simétrica e centrada em torno de zero. Ainda assim, percebe-se uma concentração de observações na região central e algumas observações extremas nas caudas, indicando pequenos desvios em relação à distribuição Normal, o que é relativamente comum em séries temporais reais.

De maneira geral, a análise gráfica sugere que o modelo apresenta um ajuste satisfatório, produzindo resíduos aproximadamente aleatórios e sem estrutura temporal evidente. A confirmação dessa conclusão é realizada por meio do teste de Ljung-Box, apresentado na seção seguinte.

6.2 Teste de Ljung-Box

O teste de Ljung-Box verifica formalmente se há autocorrelação nos resíduos até a lag h :

$$\begin{cases} H_0 : \text{os resíduos são independentes (sem autocorrelação)} \\ H_1 : \text{há autocorrelação em pelo menos um lag} \end{cases} \quad (4)$$

Box-Ljung test

```
data: residuals(modelo)
X-squared = 16.469, df = 16, p-value = 0.4207
```

Ao nível de 5%, não rejeitamos H_0 , os resíduos são independentes, indicando que o modelo capturou adequadamente a estrutura de dependência.

6.3 Normalidade dos resíduos

Shapiro-Wilk normality test

```
data: res
W = 0.96661, p-value = 1.617e-09
```

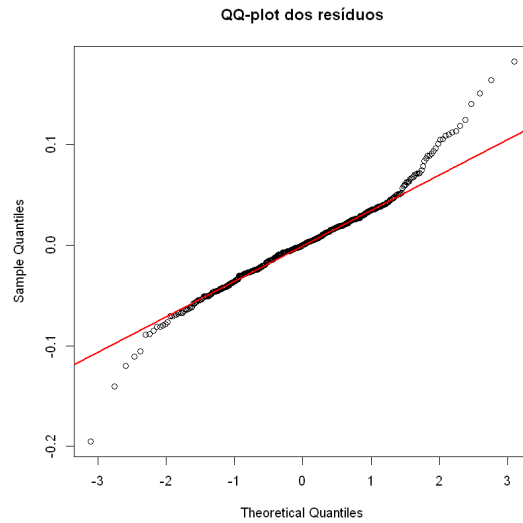


Figura 7: QQ-plot dos resíduos

Os pontos do QQ-plot não se alinham próximos à reta de referência e o p -valor do teste de Shapiro-Wilk é pequeno, então ao nível de 5% de significância rejeitamos H_0 , não há evidências de normalidade dos resíduos.

Embora o teste de normalidade tenha rejeitado a hipótese de normalidade dos resíduos, esse pressuposto não é essencial para a previsão ARIMA. O diagnóstico foi concentrado na ausência de autocorrelação residual, avaliada pelo teste de Ljung-Box.

6.4 Homocedasticidade

Agora vamos se a variância dos resíduos é constante ao longo do tempo com o teste de Breusch-Pagan:

$$\begin{cases} H_0 : \text{variância constante (homocedasticidade)} \\ H_1 : \text{variância não-constante (heterocedasticidade)} \end{cases} \quad (5)$$

studentized Breusch-Pagan test

```
data: res ~ t_index
BP = 3.2078, df = 1, p-value = 0.07329
```

H_0 não foi rejeitada, há indícios de variância constante. **Resumo do diagnóstico:**

Verificação	Ferramenta	Resultado
Independência	Ljung-Box	$p > 0,05$ — resíduos sem autocorrelação
Normalidade	Shapiro-Wilk + QQ-plot	$p < 0,05$ — não é aproximadamente normal
Homocedasticidade	Breusch-Pagan	$p > 0,05$ — variância estabilizada com log

Os resíduos não apresentaram comportamento de ruído branco perfeito quanto à distribuição (normalidade violada), embora isso possa afetar a validade dos intervalos de confiança e dos testes de significância dos coeficientes, porém a homocedasticidade é testada e o teste de Ljung-Box não indicou autocorrelação significativa. Assim, o modelo ainda é considerado adequado para **fins de previsão**.

7 Previsão

Com o modelo ajustado e os pressupostos verificados, fazemos previsões (voltando na escala original) para as próximas **52 semanas** (um ano à frente), acompanhadas dos intervalos de confiança de 80% e 95%.

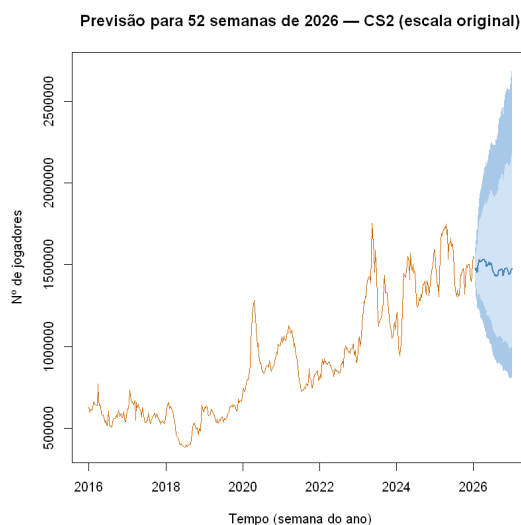


Figura 8: Previsão para as próximas 52 semanas

A previsão pontual (linha azul) segue a tendência observada, mantendo o padrão sazonal identificado. Os intervalos de confiança se alargam progressivamente com o horizonte de previsão, refletindo a incerteza crescente. Eventos imprevisíveis, atualizações, promoções ou mudanças no mercado de jogos ou até surgimento de um jogo concorrente, podem fazer com que os valores reais saiam do intervalo.

8 Validação (treino/teste)

Podemos avaliar a capacidade preditiva do modelo de forma mais rigorosa, já que no início nós usamos apenas os dados até 2025. Então naturalmente vamos usar o método hold-out(treino/teste): separamos a série em:

- **Treino:** todas as semanas até o final de 2025;
- **Teste:** as primeiras 25 semanas de 2026 (meio ano).

Ajustamos o modelo apenas nos dados de treino (até 2025) (o mesmo de antes) e comparamos as previsões com os valores reais de 2026.

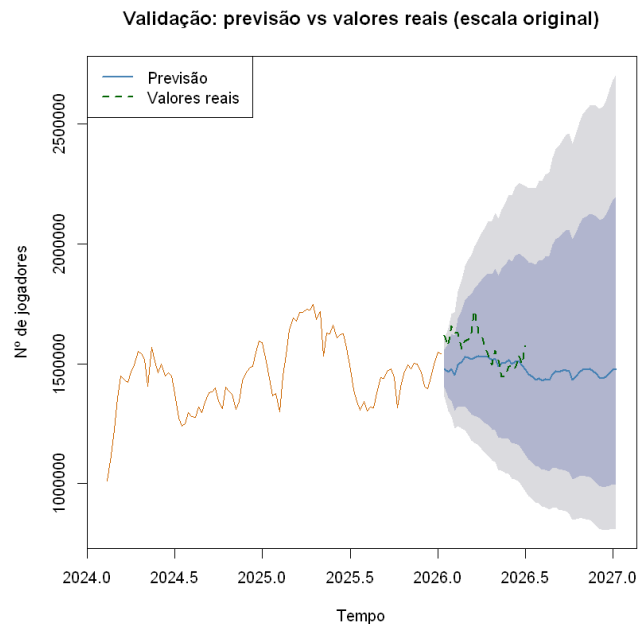


Figura 9: Validação: previsões vs. valores reais

8.1 Métricas de erro

Para quantificar o desempenho preditivo, calculamos as principais métricas:
Resultado:

MAE = 94606 jogadores
RMSE = 113451 jogadores
MAPE = 5.91 %

Métrica	Fórmula	Interpretação
MAE	$\frac{1}{h} \sum_{t=1}^h e_t $	Erro médio absoluto (mesma unidade da série)
RMSE	$\sqrt{\frac{1}{h} \sum_{t=1}^h e_t^2}$	Penaliza mais os grandes erros
MAPE	$\frac{100}{h} \sum_{t=1}^h \left \frac{e_t}{y_t} \right $	Erro percentual médio

O MAPE de 5.91 % indica, em média, as previsões erram 5.91% em relação aos valores reais. Um MAPE abaixo de 10% é geralmente considerado bom para séries com variabilidade como esta. Tanto RMSE quanto MAE indicam o número em média que as previsões erram em relação aos valores reais. O RMSE maior que o MAE indica a presença de alguns erros pontuais maiores, compatível com os picos atípicos identificados como outliers.

9 Modelos alternativos: Alisamento exponencial de Holtwinters e de ETS

Além do SARIMA ajustado sobre a série em log (itens 5 a 8 desse relatório), vale comparar o resultado com uma abordagem diferente: os modelos de **suavização exponencial**, especificamente **Holt-Winters** e **ETS** (Error, Trend, Seasonal).

Diferente do ARIMA, esses modelos não exigem que a série seja estacionária nem diferenciada, eles capturam nível, tendência e sazonalidade diretamente por meio de médias ponderadas exponencialmente. Além disso, ambos suportam sazonalidade multiplicativa nativamente, então, em vez de manter a transformação log manual, voltamos à série na escala original (número de jogadores) e deixamos o próprio modelo lidar com o crescimento da variância.

Sobre os outliers: diferentemente do SARIMA, as funções ‘HoltWinters()’ e ‘ets()’ do pacote ‘forecast’ não aceitam variáveis externas (‘xreg’), então não é possível reutilizar diretamente as dummies de outliers construídas na Etapa 4. Para não deixar os três choques identificados (COVID em 2020, o Major de 2023 e o efeito pós-Major) distorcerem o ajuste desses modelos, limpamos a série com ‘tsclean()’ antes de ajustá-los — função do próprio pacote ‘forecast’ que identifica e substitui automaticamente observações atípicas por valores interpolados, seguindo a mesma lógica por trás do ‘tsoutliers’/‘tso()’ já utilizado.

Resultados que obtivemos: Holt-Winters:

Holt-Winters exponential smoothing with trend and multiplicative seasonal comp.

```

Call:
HoltWinters(x = serie_limpa, seasonal = "multiplicative")

Smoothing parameters:
  alpha: 0.6734666
  beta : 0
  gamma: 1

Coefficients:
              [,1]
a    1.644052e+06
...
s52 9.804911e-01

SSE do ajuste: 2000020887606.4

```

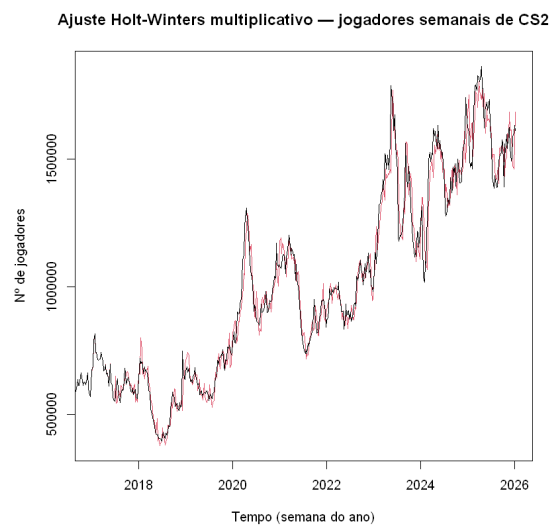


Figura 10: Alisamento com Holt-Winters

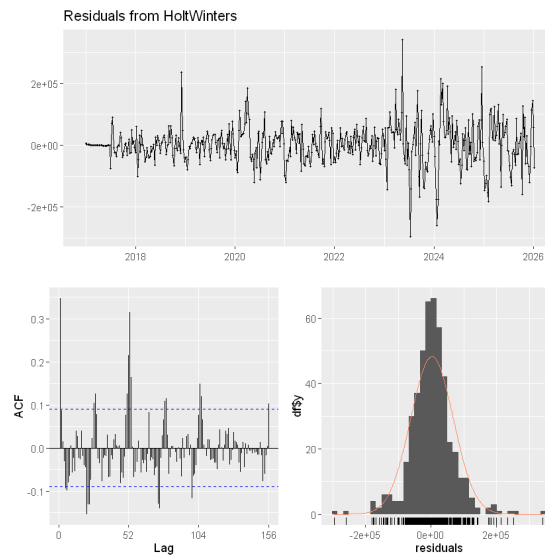


Figura 11: Diagnóstico de resíduos do modelo HW

Assim nós temos os parâmetros do modelo Holt-Winters. Sobre os resíduos, podemos ver que o modelo Holt-Winters apresenta resíduos centrados em torno de zero, porém observa-se aumento da variabilidade no final da série e diversos valores extremos. Além disso, o correlograma evidencia autocorrelações significativas, especialmente na defasagem sazonal de aproximadamente 52 semanas, indicando que o modelo não conseguiu explicar integralmente a dependência temporal dos dados.

Resultados que obtivemos: ETS:

ETS(M,N,N)

Call:

```
ets(y = serie_limpa)
```

Smoothing parameters:

alpha = 0.9999

Initial states:

l = 686650.243

sigma: 0.0525

AIC	AICc	BIC
14515.08	14515.12	14527.85

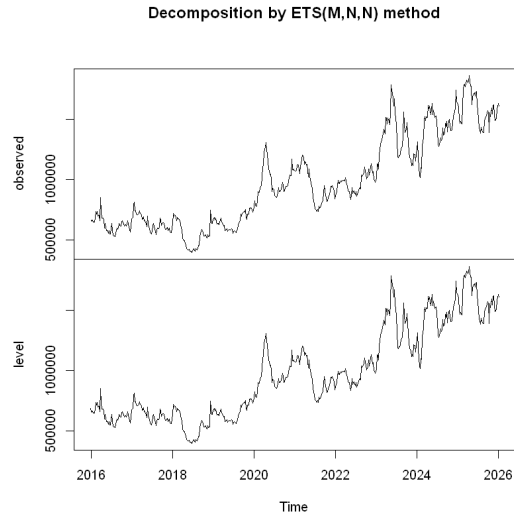


Figura 12: Decomposição por método de ETS

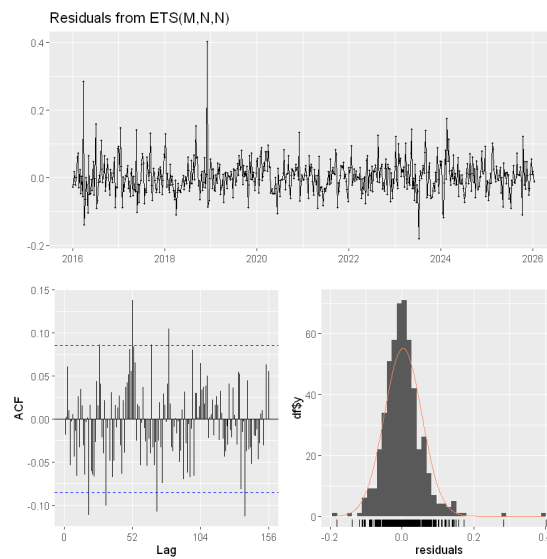


Figura 13: Diagnóstico de resíduos do modelo ETS

Assim nós temos os parâmetros do modelo ETS. Os resíduos do modelo ETS apresentam média aproximadamente nula e variância relativamente constante. Entretanto, o correlograma revela algumas autocorrelações significativas, principalmente em lags sazonais, sugerindo que parte da dependência temporal permanece nos resíduos.

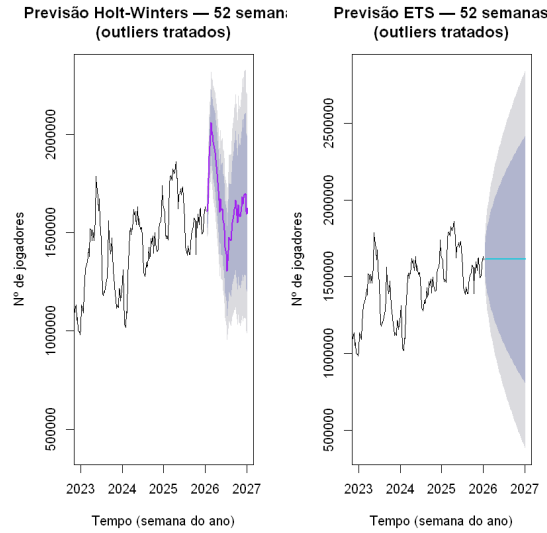


Figura 14: Previsões feitas pelo modelo Holt-Winters e ETS

A seguir podemos verificar a acurácia desses dois modelos com as métricas dentro da amostra usando a função ‘accuracy’:

Holt-Winters:

	ME	RMSE	MAE	MPE	MAPE
Test set	2838.72	65233.15	45787.11	0.02536855	4.402832

ETS:

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1
Training set	1772.845	51592	35890.21	0.03557716	3.680147	0.2007292	0.005651715

Previsões desses dois modelo para 52 semanas à frente:

Comparações:

Podemos comparar os 3 modelos usando validação treino/teste, ou seja o mesmo método que usamos anteriormente. Os resultados que obtivemos são:

Tabela 1: Comparação das métricas de previsão dos modelos avaliados.

Modelo	MAE	RMSE	MAPE (%)
SARIMA	94 606	113 451	5.91
Holt-Winters	204 309.83	241 509.35	12.97
ETS	65 892.17	83 148.60	4.33

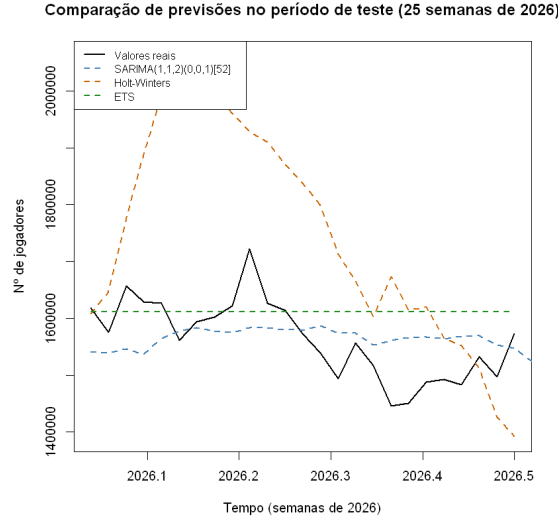


Figura 15: Visualização conjunta: valores reais vs previsão de cada modelo (período de teste)

Podemos ver que o modelo ETS apresentou melhor resultado comparando com outros 2 por errar menos na previsão. Além disso podemos ver que o modelo SARIMA superou o modelo Holt-Winters na previsão, isso pode ser a vantagem que conseguimos por tratar os outliers via ‘xreg’, o que possivelmente reduziu a distorção causadas pelos valores atípicos.

10 Conclusão

A presente análise percorreu as principais etapas de uma modelagem de séries temporais, aplicando-as ao número semanal de jogadores ativos de Counter-Strike 2 entre 2016 e 2025. Inicialmente, os dados diários extraídos da SteamDB foram agregados em médias semanais, o que permitiu suavizar ruídos diários e focar na dinâmica de médio e longo prazo. Na fase exploratória, identificou-se visualmente uma tendência, interrompida por flutuações bruscas, bem como a presença de heterocedasticidade, e uma sazonalidade anual recorrente, aspectos que foram corroborados pela decomposição STL e pela análise das funções de autocorrelação (ACF/PACF).

Para garantir a validade dos modelos paramétricos, avaliou-se a estacionariedade da série por meio dos testes ADF e KPSS. Os resultados, combinados com a inspeção gráfica, indicaram a necessidade de uma diferenciação regular de ordem $d = 1$ para tornar a série estacionária. Durante o processo, a ferramenta `tsoutliers`

revelou semanas com comportamento atípico, que puderam ser associadas a eventos relevantes do jogo, como os impactos da pandemia de COVID-19 e a realização do BLAST.tv Paris Major 2023, demonstrando como fatores exógenos afetam significativamente a quantidade de jogadores.

Com a série devidamente preparada, partiu-se para a modelagem. O modelo SARIMA(1, 1, 2)(0, 0, 1)₅₂ para a log da série foi selecionado como o mais adequado, tanto pela leitura manual dos gráficos ACF/PACF quanto pela validação automatizada fornecida pela função `auto.arima`. O diagnóstico dos resíduos indicou que o modelo é bem ajustado no que tange à ausência de autocorrelação e presença de homocedasticidade, os pressupostos mais críticos para previsões não enviesadas. No entanto, os testes de diagnóstico também apontaram desvios em relação à normalidade.

Apesar dessas limitações estatísticas, a validação preditiva realizada com os dados de 2026 revelou uma performance bastante satisfatória, com métricas de erro como o MAPE atingindo 5.91%. Isso evidencia que, mesmo com resíduos não ideais(normais), o modelo SARIMA é capaz de capturar a estrutura subjacente da série e gerar previsões confiáveis para horizontes de curto e médio prazo.

Além disso nós fizemos investigamos a possibilidade de usar modelos alternativos de alisamento exponencial (Holt-Winters e ETS). E o modelo ETS obteve melhor resultado (menor MAPE) no método de validação que usamos (treino/teste). Porém isso não significa que ETS superou o nosso modelo SARIMA, pois devemos considerar também a qualidade dos diagnósticos de resíduos.

Como desdobramento para trabalhos futuros, uma possível abordagem é tentar outras transformações ou usar outros modelos, que talvez teria um resultado melhor que o nosso modelo.

Referências

- STEAMDB. *Counter-Strike 2 – Charts*. Disponível em: <https://steamdb.info/app/730/charts/>. Acesso em: 28 jun. 2026.
- WIKIPEDIA. *Counter-Strike 2*. Disponível em: https://en.wikipedia.org/wiki/Counter-Strike_2. Acesso em: 28 jun. 2025.
- WIKIPEDIA. *BLAST.tv Paris Major 2023*. Disponível em: https://en.wikipedia.org/wiki/BLAST.tv_Paris_Major_2023. Acesso em: 28 jun. 2026.
- DE ASSIS-MOURA, MARIA SILVIA. *Material da disciplina de Séries Temporais*. Notas de aula e materiais disponibilizados no Google Classroom da disciplina, Universidade Federal de São Carlos, 2026.

- STEAM. *Counter-Strike 2*. Disponível em: https://store.steampowered.com/app/730/CounterStrike_2/. Acesso em: 28 jun. 2026.

11 Apêndice

Códigos utilizados:

```
1 library(dplyr)
2 library(lubridate)
3 library(tseries)
4 library(forecast)
5 library(lmtest)
6 library(tsoutliers)
7
8
9 cs <- read.csv("counter_strike.csv", sep = ';')
10
11 cs_semanal <- cs |>
12   dplyr::mutate(semana = floor_date(as.Date(DateTime),
13                                   unit      = "week",
14                                   week_start = 1)) |>
15   dplyr::filter(semana >= as.Date("2016-01-01"),
16                 semana <= as.Date("2025-12-31")) |>
17   dplyr::group_by(semana) |>
18   dplyr::summarise(jogadores = mean(Players, na.rm = TRUE))
19
20 cat("Dimensões:", nrow(cs_semanal), "semanas\n")
21 head(cs_semanal)
22
23 serie <- ts(cs_semanal$jogadores,
24             start      = c(2016, 1),
25             frequency = 52)
26
27 cat("Início:", start(serie), "| Fim:", end(serie),
28     "| Observações:", length(serie), "\n")
29
30 plot(serie,
31      main = "Número de jogadores semanais de Counter-Strike 2
32            (2016 2025 )",
33      xlab = "Tempo (semana do ano)",
34      ylab = "N de jogadores",
35      col  = "darkorange3")
```

```

36 summary(serie)
37 decomp <- stl(serie, s.window = "periodic")
38 plot(decomp,
39      main = "Decomposição STL da série de jogadores de CS2",
40      col = "darkorange3")
41
42 par(mfrow = c(1, 2))
43 acf(serie, main = "Autocorrelação (ACF)", col = "
44      darkorange3")
45 pacf(serie, main = "Autocorrelação Parcial (PACF)", col = "
46      darkorange3")
47 par(mfrow = c(1, 1))
48
49 adf.test(serie) # H0: não-estacionária
50 kpss.test(serie) # H0: estacionária
51 serie_diff <- diff(serie)
52
53 plot(serie_diff,
54      main = "Série diferenciada (ordem 1)",
55      xlab = "Semanas",
56      ylab = "Variação no n de jogadores",
57      col = "darkorange3")
58
59 adf.test(serie_diff)
60 kpss.test(serie_diff)
61 par(mfrow = c(1, 2))
62 acf(serie_diff, main = "ACF série diferenciada", col =
63      "darkorange3", lag.max = 60)
64 pacf(serie_diff, main = "PACF série diferenciada", col =
65      "darkorange3", lag.max = 60)
66 par(mfrow = c(1, 1))
67
68 library(tsoutliers)
69 #outliers3 <- tso(serie)
70
71 outliers3 <- data.frame( # fixando outliers sem ter que
72      rodar de novo o pacote tsoutliers
73      type = c("LS", "TC", "LS"),
74      ind = c(220, 384, 391),
75      stringsAsFactors = FALSE
76 )
77
78 par(mfrow = c(1, 2))
79 acf(serie_diff, main = "ACF série diferenciada", col =

```

```

    "darkorange3", lag.max = 60)
74 pacf(serie_diff, main = "PACF      série diferenciada", col =
    "darkorange3", lag.max = 60)
75 par(mfrow = c(1, 1))
76
77 # extrai a matriz de dummies dos outliers detectados
78 dummies <- outliers.effects(outliers3, n = length(serie))
79 # ajusta o ARIMA já considerando os outliers
80 #modelo_ajustado <- auto.arima(serie, xreg = dummies)
81
82 #modelo_ajustado <- Arima(serie, order = c(0,0,1),
83 #   seasonal = list(order = c(0,0,1),period = 52),
84 #   xreg = dummies,
85 #)
86 serie <- log(serie)
87 serie_diff <- diff(serie)
88 plot(serie_diff,
89     main = "Série em log diferenciada (ordem 1)",
90     xlab = "Semanas",
91     ylab = "Variação do log(n   de jogadores)",
92     col = "darkorange3")
93
94 par(mfrow = c(1, 2))
95 acf (serie_diff, main = "ACF      série diferenciada", col =
    "darkorange3", lag.max = 60)
96 pacf(serie_diff, main = "PACF      série diferenciada", col =
    "darkorange3", lag.max = 60)
97 par(mfrow = c(1, 1))
98
99 #modelo <- auto.arima(serie, stepwise = FALSE)
100 modelo <- Arima(serie, order = c(1,1,2),
101     seasonal = list(order = c(0,0,1),period = 52),xreg =
        dummies)
102 summary(modelo)
103
104 checkresiduals(modelo, col = "darkorange3")
105
106 Box.test(residuals(modelo), lag = 20, type = "Ljung-Box",
        fitdf = sum(modelo$arima[1:4]))
107
108 res <- residuals(modelo)
109
110 # QQ-plot

```

```

111 qqnorm(res, main = "QQ-plot dos resíduos")
112 qqline(res, col = "red", lwd = 2)
113
114 # Teste de Shapiro-Wilk
115 shapiro.test(res)
116
117 t_index <- seq_along(res)
118 bptest(res ~ t_index)
119
120 h <- 52 # horizonte de previsão (semanas)
121
122 xreg_futuro <- matrix(0, nrow = h, ncol = ncol(dummies))
123 colnames(xreg_futuro) <- colnames(dummies)
124
125 prev <- forecast(modelo, h = h, xreg = xreg_futuro) #
    previsão na escala log
126
127 # Reversão para a escala original (n de jogadores), via exp
    (), para facilitar a leitura
128 prev_original <- prev
129 prev_original$mean <- exp(prev$mean)
130 prev_original$lower <- exp(prev$lower)
131 prev_original$upper <- exp(prev$upper)
132 prev_original$x <- exp(prev$x)
133
134 plot(prev_original,
135       main = paste("Previsão para", h, "semanas de 2026
    CS2 (escala original)"),
136       xlab = "Tempo (semana do ano)",
137       ylab = "N de jogadores",
138       col = "darkorange3",
139       fcol = "steelblue",
140       shadecols = c("#a8c8e8", "#d0e4f5"),
141       flwd = 2)
142
143
144 cs <- read.csv("steamdb_chart_730.csv", sep = ';')
145
146 cs_semanal_todos <- cs |>
147   dplyr::mutate(semana = floor_date(as.Date(DateTime),
148                                   unit = "week",
149                                   week_start = 1)) |>
150   dplyr::filter(semana >= as.Date("2016-01-01"),

```

```

151         semana <= as.Date("2026-06-24")) |>
152     dplyr::group_by(semana) |>
153     dplyr::summarise(jogadores = max(Players, na.rm = TRUE)) #
        os dados de 2026 ainda não estão completos, então peguei
        o máximo de cada semana como uma aproximação para não
        perder nenhuma semana
154
155     cat("Dimensões:", nrow(cs_semanal_todos), "semanas\n")
156     tail(cs_semanal_todos)
157     serie_total <- ts(cs_semanal_todos$jogadores,
158                       start      = c(2016, 1),
159                       frequency  = 52)
160
161     # mesma transformação logarítmica aplicada à série completa (
        treino + teste)
162     serie_total <- log(serie_total)
163
164     cat("Início:", start(serie_total), "| Fim:", end(serie_total)
165         ,
        "| Observações:", length(serie_total), "\n")
166
167     n_total <- length(serie_total)
168     n_teste  <- 25 # as primeiras 25 semanas de 2026
169     n_treino <- n_total - n_teste
170
171     serie_treino <- head(serie_total, n_treino) # em log
172     serie_teste  <- tail(serie_total, n_teste)  # em log
173
174     # iremos usar o mesmo modelo já que ele foi ajustado para a sé
        rie completa em log, mas vamos apenas fazer a previsão
        para as primeiras 25 semanas de 2026
175
176
177     # Previsão para o período de teste (na escala log)
178     prev_teste <- forecast(modelo, h = n_teste, xreg = xreg_
        futuro)
179
180     # Reversão para a escala original antes de plotar/comparar
181     prev_teste_original <- prev_teste
182     prev_teste_original$mean <- exp(prev_teste$mean)
183     prev_teste_original$lower <- exp(prev_teste$lower)
184     prev_teste_original$upper <- exp(prev_teste$upper)
185     prev_teste_original$x <- exp(prev_teste$x)

```

```

186
187 serie_teste_original <- exp(serie_teste)
188
189 # Visualização: série real vs previsão (n de jogadores)
190 plot(prev_teste_original,
191       main = "Validação: previsão vs valores reais (escala
192             original)",
193       xlab = "Tempo",
194       ylab = "N de jogadores",
195       col = "darkorange3",
196       fcol = "steelblue",
197       flwd = 2)
198 lines(serie_teste_original, col = "darkgreen", lwd = 2, lty =
199       2)
200 legend("topleft",
201       legend = c("Previsão para o ano de 2026 inteiro", "
202             Valores reais (2026)"),
203       col = c("steelblue", "darkgreen"),
204       lty = c(1, 2), lwd = 2)
205
206
207 plot(prev_teste_original,
208       include = 100,
209       main = "Validação: previsão vs valores reais (escala
210             original)",
211       xlab = "Tempo",
212       ylab = "N de jogadores",
213       col = "darkorange3",
214       fcol = "steelblue",
215       flwd = 2)
216 lines(serie_teste_original, col = "darkgreen", lwd = 2, lty =
217       2)
218 legend("topleft",
219       legend = c("Previsão", "Valores reais"),
220       col = c("steelblue", "darkgreen"),
221       lty = c(1, 2),
222       lwd = 2)
223
224 # Erros calculados na escala original (n de jogadores), após
225   s reverter o log com exp()
226 erros <- as.vector(serie_teste_original) - as.vector(prev_

```

```

    teste_original$mean)
223
224 MAE <- mean(abs(erros))
225 RMSE <- sqrt(mean(erros^2))
226 MAPE <- mean(abs(erros / as.vector(serie_teste_original))) *
    100
227
228 cat(sprintf("MAE = %10.0f jogadores\n", MAE))
229 cat(sprintf("RMSE = %10.0f jogadores\n", RMSE))
230 cat(sprintf("MAPE = %10.2f %%\n", MAPE))
231
232 # Revertendo para a escala original: Holt-Winters e ETS com
    componente sazonal
233 # multiplicativo já lidam internamente com a variância
    crescente, então não
234 # precisamos manter a transformação log manual aqui.
235 serie_original <- exp(serie)
236
237 # Como HoltWinters()/ets() não aceitam xreg para tratar
    outliers (diferente do
238 # SARIMA), "limpamos" a série antes do ajuste com tsclean(),
    que identifica e
239 # substitui observações atípicas por valores interpolados.
240 serie_limpa <- tsclean(serie_original)
241
242 hw_fit <- HoltWinters(serie_limpa, seasonal = "multiplicative
    ")
243 hw_fit
244 # Parâmetros estimados (alpha, beta, gamma) e SSE do ajuste
245 hw_fit$alpha; hw_fit$beta; hw_fit$gamma
246 hw_fit$$SSE
247
248 plot(hw_fit,
249     main = "Ajuste Holt-Winters multiplicativo jogadores
        semanais de CS2",
250     xlab = "Tempo (semana do ano)",
251     ylab = "N de jogadores")
252 library(forecast)
253
254 # Ajuste automático do modelo ETS sobre a série limpa (a funç
    ão escolhe a
255 # melhor combinação Erro/Tendência/Sazonalidade por AICc)
256 ets_fit <- ets(serie_limpa)

```

```

257 summary(ets_fit)
258 plot(ets_fit)
259 checkresiduals(ets_fit)
260 acc_hw <- accuracy(fitted(hw_fit)[, 1], serie_limpa[-(1:52)
    ]) # HoltWinters só tem ajuste a partir do 1 ciclo
    completo
261 acc_ets <- accuracy(ets_fit)
262
263 acc_hw
264 acc_ets
265 checkresiduals(hw_fit)
266 fc_hw <- forecast(hw_fit, h = 52, level = c(80, 95))
267 fc_ets <- forecast(ets_fit, h = 52, level = c(80, 95))
268
269 fc_hw
270 fc_ets
271 par(mfrow = c(1, 2))
272 plot(fc_hw, main = "Previsão Holt-Winters 52 semanas\n(
    outliers tratados)", fcol = "darkorange3",
273 ylab = "N de jogadores", xlab = "Tempo (semana do ano)
    ")
274 plot(fc_ets, main = "Previsão ETS 52 semanas\n(outliers
    tratados)",
275 ylab = "N de jogadores", xlab = "Tempo (semana do ano)
    ")
276 par(mfrow = c(1, 1))
277 par(mfrow = c(1, 2))
278 plot(fc_hw, xlim = c(2023, 2027),
279 main = "Previsão Holt-Winters 52 semanas\n(outliers
    tratados)", fcol = "#a020f0",
280 ylab = "N de jogadores", xlab = "Tempo (semana do ano)
    ")
281 plot(fc_ets, xlim = c(2023, 2027),
282 main = "Previsão ETS 52 semanas\n(outliers tratados)
    ", fcol = "#08cbdddc",
283 ylab = "N de jogadores", xlab = "Tempo (semana do ano)
    ")
284 par(mfrow = c(1, 1))
285 # Série de treino/teste na escala original, usando a mesma
    janela da Etapa 8
286 serie_treino_original <- tsclean(exp(serie_treino))
287 serie_teste_original2 <- exp(serie_teste) # equivalente ao
    serie_teste_original já calculado na Etapa 8

```

```

288
289 # Reajuste de Holt-Winters e ETS apenas com os dados de
    treino
290 hw_fit_treino <- HoltWinters(serie_treino_original, seasonal
    = "multiplicative")
291 ets_fit_treino <- ets(serie_treino_original)
292
293 # Previsão para o mesmo horizonte de teste (n_teste = 25
    semanas)
294 fc_hw_teste <- forecast(hw_fit_treino, h = n_teste)
295 fc_ets_teste <- forecast(ets_fit_treino, h = n_teste)
296 # Erros de cada modelo no período de teste (escala original,
    n de jogadores)
297 erros_hw <- as.vector(serie_teste_original) - as.vector(fc_
    hw_teste$mean)
298 erros_ets <- as.vector(serie_teste_original) - as.vector(fc_
    ets_teste$mean)
299 # 'erros' (SARIMA) já foi calculado na Etapa 8
300
301 metricas <- function(erros, real) {
302   c(MAE = mean(abs(erros)),
303     RMSE = sqrt(mean(erros^2)),
304     MAPE = mean(abs(erros / real)) * 100)
305 }
306
307 tabela_comparativa <- rbind(
308   SARIMA = metricas(erros, as.vector(serie_teste_
    original)),
309   HoltWinters = metricas(erros_hw, as.vector(serie_teste_
    original)),
310   ETS = metricas(erros_ets, as.vector(serie_teste_
    original))
311 )
312
313 round(tabela_comparativa, 2)
314 # Visualização conjunta: valores reais vs previsão de cada
    modelo (período de teste)
315 plot(serie_teste_original, type = "l", col = "black", lwd =
    2, lty = 1,
316   main = "Comparação de previsões no período de teste (25
    semanas de 2026)",
317   xlab = "Tempo (semanas de 2026)",
318   ylab = "N de jogadores",

```

```

319     ylim = range(c(serie_teste_original,
320                   prev_teste_original$mean,
321                   fc_hw_teste$mean,
322                   fc_ets_teste$mean)))
323
324 lines(prev_teste_original$mean, col = "steelblue", lwd = 2,
325       lty = 2)
326 lines(fc_hw_teste$mean, col = "darkorange3", lwd = 2,
327       lty = 2)
328 lines(fc_ets_teste$mean, col = "forestgreen", lwd = 2,
329       lty = 2)
330
331 legend("topleft",
332       legend = c("Valores reais", "SARIMA(1,1,2)(0,0,1)[52]",
333                 , "Holt-Winters", "ETS"),
334       col = c("black", "steelblue", "darkorange3", "
335               forestgreen"),
336       lty = c(1, 2, 2, 2), lwd = 2, cex = 0.8)

```